

Chapter 10. Low-Rank Methods

In the preceding chapter, we lamented at the challenge of working with so many features. By letting each item be its own feature, we were able to express a lot of information about user preference and item-affinity correlations, but we were in big trouble in terms of the curse of dimensionality. Combine this with the reality of very sparse features, and you're in danger. In this chapter, we'll turn to smaller feature spaces. By representing users and items as low-dimensional vectors, we can capture the complex relationships between them in a more efficient and effective way. This allows us to generate more personalized and relevant recommendations for users while also reducing the computational complexity of the recommendation process.

We will explore the use of low-dimensional embeddings and discuss the benefits and some of the implementation details of this approach. We will also look at code in JAX that uses modern gradient-based optimization to reduce the dimension of your item or user representations.

Latent Spaces

You are already familiar with feature spaces, which are usually categorical or vector-valued direct representations of the data. This can be the raw red, green, and blue values of an image, counts of items in a histogram, or attributes of an object like length, width, and height. Latent features, on the other hand, do not represent any specific real value feature of the items but are initialized randomly and then learned to suit a task. The GloVe embeddings we discussed in [Chapter 8](#) are one such example of a latent vector that was learned to represent the log count of words. Here we will cover more ways to generate these latent features or embeddings.

This chapter relies heavily on linear algebra, so it's good to read up on vectors, dot products, and norms of vectors before proceeding. Having an understanding of matrices and the rank of matrices will also be useful. Consider [*Linear Algebra and Its Applications*](#) by Gilbert Strang.

One of the reasons latent spaces are so popular is that they are usually lower in dimension than the features they represent. For example, if the user-item rating matrix or interaction matrix (where the matrix entries are 1 if a user has interacted with an item) is $N \times M$ dimensional, then factorizing the matrix into latent factors of $N \times K$ and $K \times M$, where K is much smaller than N or M is an approximation of the missing entries because we're relaxing the factorization. K being smaller than N or M is usually called an *information bottleneck*—that is, we are forcing the matrix to be made up of a much smaller matrix. This means the ML model has to make up missing entries, which might be good for recommender systems. As long as users have interacted with enough similar items, by forcing the system to have a lot less capacity in terms of degrees of freedom, then factorization can completely reconstruct the matrix, and the missing entries tend to get filled by similar items.

Let's see what happens, for example, when we factor a user-item matrix of 4×4 into a 4×2 and a 2×4 vector using SVD.

We are supplying a matrix whose rows are users and whose columns are items. For example, row 0 is `[1, 0, 0, 1]`, which means user 0 has selected item 0 and item 3. These can be ratings or purchases. Now let's look at some code:

```
import numpy as np

a = np.array([
    [1, 0, 0, 1],
    [1, 0, 0, 0],
    [0, 1, 1, 0],
    [0, 1, 0, 0]
])
```

```

u, s, v = np.linalg.svd(a, full_matrices=False)

# Set the last two eigenvalues to 0.
s[2:4] = 0
print(s)
b = np.dot(u * s, v)
print(b)

# These are the eigenvalues with the smallest two set to 0.
s = [1.61803399 1.61803399 0.          0.          ]

# This is the newly reconstructed matrix.
b = [[1.17082039 0.          0.          0.7236068 ]
      [0.7236068 0.          0.          0.4472136 ]
      [0.          1.17082039 0.7236068 0.          ]
      [0.          0.7236068 0.4472136 0.          ]]

```

Notice that the user in row 1 now has a score for an item in column 3, and the user in row 3 now has a positive score for the item in column 2. This phenomenon is generally known as *matrix completion* and is a good property for recommender systems to have because now we get to recommend new items to users. The general method of forcing the ML to go through a bottleneck that is smaller than the size of the matrix that it is trying to reconstruct is known as a *low-rank approximation* because the rank of the approximation is 2 but the rank of the original user-item matrix is 4.

WHAT IS THE RANK OF A MATRIX?

An $N \times M$ matrix may be considered as N row vectors (corresponding to users) and M column vectors (corresponding to items). When you consider the N vectors of dimension M , the *rank of the matrix* is the volume of the polyhedron defined by the N vectors in M dimensions. This is often different from the way we talk about the rank of matrices, however. While it's the most natural and precise definition, we instead say it's the “minimum number of dimensions necessary to represent the vectors of the matrix.”

We will cover SVD in more detail later in the chapter. This was just to whet your appetite to understand how latent spaces are related to recommender systems.

Dot Product Similarity

In [Chapter 3](#) we introduced similarity measures, but now we return to the dot product in the context of similarity because of their increased importance in latent spaces. After all, latent spaces are built on the assumption that distance is similarity.

The dot-product similarity is meaningful in recommendation systems because it provides a geometric interpretation of the relationship between users and items in the latent space (or potentially items and items, users and users, etc.). In the context of recommendation systems, the dot product can be seen as a projection of one vector onto another, indicating the degree of similarity or alignment between the user's preferences and the item's characteristics.

To understand the geometric significance of the dot product, consider two vectors, u and p , representing the user and the product in the latent space, respectively. The dot product of these two vectors can be defined as follows:

$$u \times p = ||u|| ||p|| \cos(\theta)$$

Here, $||u||$ and $||p||$ represent the magnitudes of the user and product vectors, and θ is the angle between them. The dot product is thus a measure of the projection of one vector onto another, which is scaled by the magnitudes of both vectors.

The cosine similarity, which is another popular similarity measure in recommendation systems, is derived directly from the dot product:

$$\text{cosine similarity}(u, p) = \frac{(u \times p)}{(||u|| ||p||)}$$

The cosine similarity ranges from -1 to 1 , where -1 indicates completely dissimilar preferences and characteristics. A 0 indicates no similarity, and 1 indicates perfect alignment between the user's preferences and the product's characteristics. In the context of recommendation systems, cosine similarity provides a normalized measure of similarity that is invariant to the magnitudes of the user and product vectors. Note that the

choice of using cosine similarity versus L2 distance depends on the type of embeddings you're using and the way you optimize the computations. In practice, the only important feature is often the relative values.

The geometric interpretation of the dot product (and cosine similarity) in recommendation systems is that it captures the alignment between user preferences and product characteristics. If the angle between the user and product vectors is small, the user's preferences align well with the product's characteristics, leading to a higher similarity score. Conversely, if the angle is large, the user's preferences and product's characteristics are dissimilar, resulting in a lower similarity score. By projecting user and item vectors onto each other, the dot-product similarity can capture the degree of alignment between user preferences and item characteristics, allowing the recommendation system to identify items that are most likely to be relevant and appealing to the user.

Anecdotally, the dot product seems to capture popularity, as very long vectors tend to be easy to project on anything that isn't completely perpendicular or pointing away from them. As a result, a trade-off exists between frequently recommending popular items with a large vector length and longer-tail items that have smaller angular difference with cosine distance.

[Figure 10-1](#) considers two vectors, a and b . With cosine similarity, the vectors are unit length, so the angle is just the measure of similarity.

However, with dot product, a very long vector like c might be considered more similar to a than b even though the angle between a and b is smaller because of the longer length of c . These long vectors tend to be very popular items that co-occur with many other items.

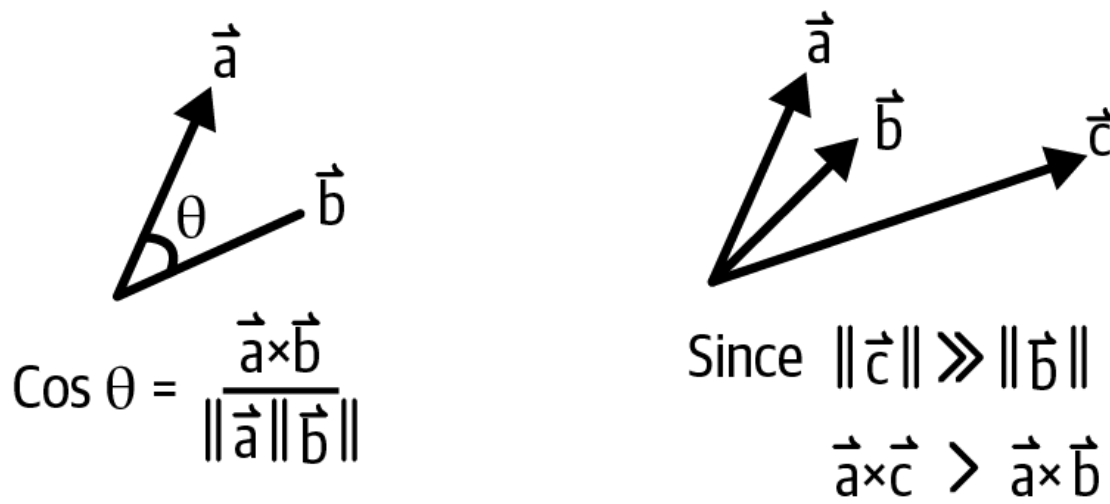


Figure 10-1. Cosine versus dot-product similarity

Co-occurrence Models

In our Wikipedia co-occurrences examples, we determined that the co-occurrence structure between two items could be used to generate measures of similarity. We covered how PMI can take the counts of co-occurrence and make recommendations based on very high mutual information between an item in the cart and others.

As we've discussed, PMI is not a distance metric but still has important similarity measures based on co-occurrence. Let's return to this topic.

Recall from earlier that PMI is defined as follows:

$$\frac{p(x_i, x_j)}{p(x_i) * p(x_j)} = \frac{C_{\mathcal{J}}(x_i, x_j) * \# \text{ total interactions}}{\#(x_i) * \#(x_j)}$$

Now let's consider the *discrete co-occurrence distribution*, CD_{x_i} , defined as the collection of co-occurrences over all other x_j :

$$CD_{x_i} = C_{\mathcal{J}}(x_i, x_1), \dots, C_{\mathcal{J}}(x_i, x_i), \dots, C_{\mathcal{J}}(x_i, x_N)$$

Here, $j \in 1 \dots N$, and N is the total number of items. This represents the co-occurrence histogram between x_i and all other items. By introducing this discrete distribution, we can utilize another tool: the Hellinger distance.

We can measure distributional distance in a few ways, each with different advantages. For our discussion, we will not go deeply into the differ-

ences and will stick to the simplest but most appropriate. Hellinger distance is defined as follows:

$$H(P, Q) = \sqrt{1 - \sum_i^n \sqrt{p_i q_i}} = \frac{1}{\sqrt{2}} \|\sqrt{P} - \sqrt{Q}\|_2$$

$P = \langle p_i \rangle$ and $Q = \langle q_i \rangle$ are two probability density vectors. In our setting, P and Q can be CD_{x_i} and CD_{x_j} .

The motivation behind this process is that we now have a proper distance between items purely based on co-occurrences. We can use any dimension transformation or reduction on this geometry. Later we will show dimension-reduction techniques that can use an arbitrary distance matrix and reduce the space to a lower-dimensional embedding that approximates it.

WHAT ABOUT MEASURE SPACES AND INFORMATION THEORY?

While we're discussing distributions, you may find yourself wondering, "Is there a distance between distributions such that distributions are points in a latent space?" Oh, you weren't wondering that? Well, OK. We'll address it anyway.

The short answer is that we can measure the differences between distributions. The most popular is Kullback–Leibler (KL) divergence, which is usually described in a Bayesian sense as the amount of surprise in seeing the distribution P , when expecting the distribution Q . However, KL is not a proper distance metric because it is asymmetric.

Another symmetric distance metric that has some nice properties is the Hellinger distance. Hellinger distance is effectively the 2-norm measure theoretic distance. Additionally, Hellinger distance naturally generalizes to discrete distributions.

If this still hasn't scratched your itch for abstraction, we can also consider the total variation distance, which is the limit in the space of Fisher's exact distance measures, which really means that it has all the nice properties of a distance of two distributions and no measure would ever consider them more dissimilar. Well, all the nice properties except for one: it's not smooth. If you also want smoothness for differentiability, you'll need to approximate it via an offset.

If you ever need a distance between distributions, just use Hellinger.

Reducing the Rank of a Recommender Problem

We've shown that as the number of items and users grows, we rapidly increase the dimensionality of our recommender problem. Because we're representing each item and user as a column or vector, this scales like n^2 . One way to push back against this difficulty is by rank reduction; recall our previous discussions about rank reduction via factorization.

Like many integers, many matrices can be *factored* into smaller matrices; for integers, *smaller* means of smaller value, and for matrices, *smaller* means of smaller dimensions. When we factor an $N \times M$ matrix, we will be looking for two matrices $U_{N \times d}$ and $V_{d \times M}$; note that when you multiply matrices together, they must share a dimension, and that dimension is eliminated, leaving the other two. Here, we'll consider MFs when $d \leq N$ and $d \leq M$. By factorizing a matrix, we ask for two matrices that together equal, or approximate, the original matrix:

$$A_{i,j} \simeq U_i \cdot V_j$$

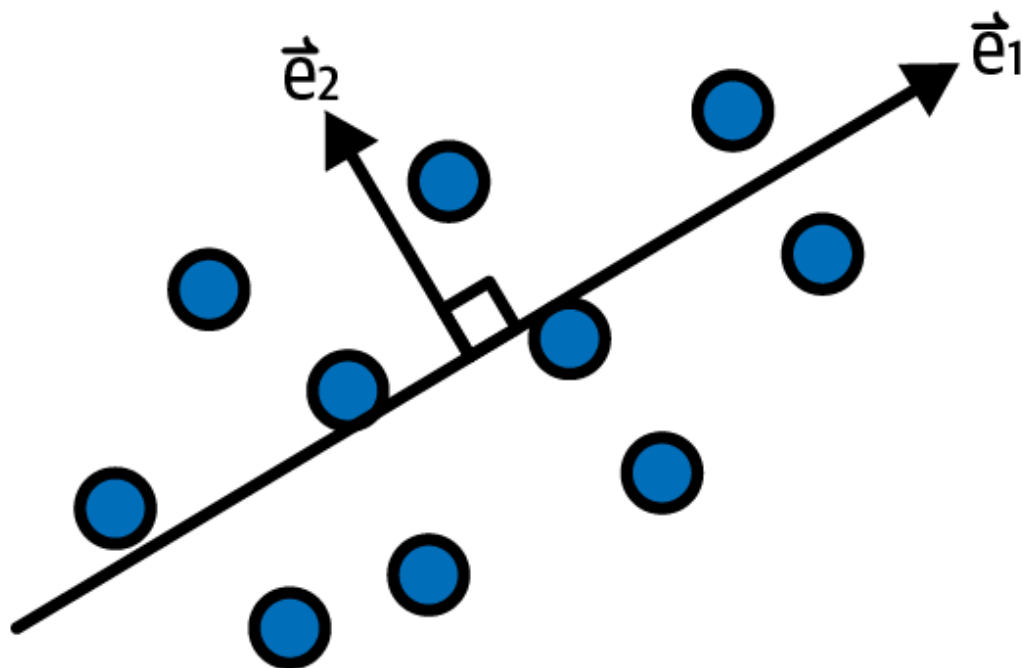
We seek a small value for d to reduce the number of latent dimensions. As you may have already noticed, each of the matrices $U_{N \times d}$ and $V_{d \times M}$ will correspond to rows or columns of the original ratings matrix. However, they're expressed in fewer dimensions. This utilizes the idea of a *low-dimensional latent space*. Intuitively, a latent space seeks to represent the same relationships as the full $N \times M$ dimensional relationships in two sets of relationships: items versus latent features, and users versus latent features.

These methods are also popular in other kinds of ML, but for our case, we'll primarily be looking at factorizing ratings or interaction matrices.

SVD and MF are closely related, but SVD is an important special case. The key difference is in how the factorization is done and the types of matrices that each can be applied to.

[Figure 10-2](#) shows how SVD works. The eigenvectors $\vec{e}_1$ and $\vec{e}_2$ correspond to the largest two eigenvalues. $\vec{e}_1$ explains more of the data than $\vec{e}_2$ and lies along the direction of the largest spread of points.

Eigenvectors are always perpendicular to each other, so their dot product is always 0.



$\vec{e}_1$ explains more of the data than $\vec{e}_2$

$\vec{e}_1$ is perpendicular to $\vec{e}_2$

Figure 10-2. Singular value decomposition

SVD is a specific type of MF that decomposes a matrix into three separate matrices: a left singular matrix, a diagonal matrix, and a right singular matrix. SVD can be applied to any real-valued matrix, but it is particularly well suited to dense matrices with many nonzero entries; additionally, SVD matrices have properties useful for extracting particular kinds of relationships between latent features. The columns and rows of the singular matrices are the eigenvectors, and the values in the diagonal ma-

trix are the eigenvalues. This decomposition is handy in seeing how much information in the original space is explained by the eigenvectors and corresponds to the magnitude of the eigenvalue, so an eigenvector with a larger eigenvalue explains more of the original data than an eigenvector with a correspondingly smaller eigenvalue.

MF decomposes a user-item matrix into two matrices that represent the preferences of users and the characteristics of items. This allows the recommendation system to generate personalized recommendations by matching the preferences of users with the characteristics of items.

Frequently, you must overcome a few challenges when considering MF:

- The matrix you wish to factor is sparse and often non-negative and/or binary.
- The number of nonzero elements in each item vector can vary wildly, as we saw in the Matthew effect.
- Factorizing matrices is cubic in complexity.
- SVD and other full-rank methods don't work without imputation, which itself is complicated.

We'll address these with some alternative optimization methods.

Optimizing for MF with ALS

The basic optimization we wish to execute is to approximate as follows:

$$A_{i,j} \simeq U_i \cdot V_j$$

Notably, if you wish to optimize matrix entries directly, you'll need to optimize $d^2 \cdot N \cdot M$ elements simultaneously corresponding to the numbers of parameters in these factorizations. We can easily achieve a significant speedup, however, by alternating between tuning one matrix or the other. This is called *alternating least squares*, commonly *ALS*, and it is a common approach to this problem. Instead of back-propagating updates to all terms in both matrices on each pass, you may update only one of the two matrices, which dramatically reduces the number of computations that need to take place.

ALS seeks to switch back and forth between U and V , evaluating on the same loss function but updating the weights in only one matrix at a time:

$$\begin{aligned}U &\leftarrow U - \eta * U * \nabla U * \mathcal{D} A, UV) \\V &\leftarrow V - \eta * V * \nabla V * \mathcal{D} A, UV)\end{aligned}$$

Here, η is the learning rate and $\mathcal{D}$ is our chosen distance function. We'll present more details of this distance function momentarily. Before we move on, let's consider a few of the intricacies here:

- Each of these update rules requires the gradient with respect to the relevant factor matrix.
- We update an entire factor matrix at a time, but we evaluate loss on the product of the factor matrices versus the original matrix.
- We have a mysterious distance function.
- By the way that we've constructed this optimization, we're implicitly assuming that we'll use this process to converge two well-approximating matrices (we often also impose a limit on the number of iterations).

In JAX, these optimizations will be straightforward to implement, and we'll see how similar the equational forms and the JAX code look.

DISTANCE BETWEEN MATRICES

We can determine the distance between two matrices in a variety of ways. As we've seen before, different distance measurements for vectors yield different interpretations from the underlying space. We won't have as many complications for these computations, but it's worth a short observation. The most obvious approach is one you've already seen, *observed mean squared error*:

$$\frac{\sum_{\Omega} (A_{i,j} - U_i \cdot V_j)^2}{|\Omega|}$$

One useful alternative to the observed mean squared error can be used when you have a single nonzero entry for a user vector (alternatively, a max rating). In that case, you could instead use a cross-entropy loss, which provides a *logistic MF*, and thus a probability estimate. For more details on how to implement this, see the [“Matrix Factorization for Recommender Systems” tutorial](#) by Kyle Chung.

In our observed ratings, we expect (and see!) a large number of missing values and some item vectors with an overrepresented number of ratings. This suggests that we should consider nonuniformly weighted matrices. Next we'll discuss how to account for this and other variants with regularization.

Regularization for MF

Weighted alternating least squares (WALS) is similar to ALS but attempts to resolve these two data issues more gracefully. In WALS, the weight assigned to each observed rating is inversely proportional to the number of observed ratings for the user or item. Therefore, observed ratings for users or items with few ratings are given more weight in the optimization process.

We can apply these weights as a regularization parameter in our eventual loss function:

$$\frac{\sum_{\Omega} A_{i,j} - \langle U_i, V_j \rangle)^2}{|\Omega|} + \frac{1}{N} \sum |U|$$

Other regularization methods are important, and also popular, for MF. We'll discuss these two powerful regularization techniques:

- Weight decay
- Gramian regularization

As is often the case, *weight decay* is our l^2 regularization, which in this case is at the level of the Frobenius norm, i.e., the magnitude of the weight matrix. An elegant way to view this weight decay is that it's minimizing the magnitude of the *singular values*.

Similarly, MF has another regularization technique that looks very standard but is quite different in calculation. This is via the *Gramians*—essentially regularizing the size of the individual matrix entries, but there's an elegant trick for the optimization. In particular, a Gramian of a matrix U is the product $U^T U$. The eagle-eyed among you may recognize this term as the same term we previously used to calculate co-occurrences for binary matrices. The connection is that both are simply trying to find effi-

cient representations of dot products between a matrix's rows and columns.

These regularizations are the Frobenius terms:

$$R(U, V) = \frac{1}{N} \sum_i \left\| U_i \right\|_2^2 + \frac{1}{M} \sum_j \|V_j\|_2^2$$

Or expanded, the equation looks like this:

$$R(U, V) = \frac{1}{N} \sum_i \sum_k^d U_{i,k}^2 + \frac{1}{M} \sum_j \sum_l^d V_{j,l}^2$$

And here are the Gramian terms:

$$\begin{aligned} G(U, V) &:= \frac{1}{N \cdot M} \sum_i^N \sum_j^M \langle U_i, V_j \rangle^2 \\ &= \frac{1}{N \cdot M} * \sum_{k,l}^d U^T U * V^T V)_{k,l} \end{aligned}$$

Finally, we have our loss function:

$$\frac{1}{|\Omega|} \sum_{(i,j) \in \Omega} (A_{ij} - \langle U_i, V_j \rangle)^2 + \lambda_R \frac{1}{N} \sum_i \sum_k^d U_{i,k}^2 + \frac{1}{M} \sum_j \sum_l^d V_{j,l}^2 \Bigg) + \lambda_G \frac{1}{N \cdot M} * \sum_{k,l}^d U^T U * V^T V)_{k,l} \Bigg)$$

Regularized MF Implementation

So far, we've written a *lot* of math symbols, but all of those symbols have allowed us to arrive at a model that is extremely powerful. *Regularized matrix factorization* is an effective model for medium-sized recommender problems. This model type is still in production for many serious businesses. One classic issue with MF implementations is performance, but because we're working in JAX, which has extremely native GPU support, our implementation can actually be much more compact than what you may find in something like a [PyTorch example](#).

Let's work through how this model would look to predict ratings for a user-item matrix via this doubly regularized model with Gramians.

First we'll do the simple setup. This will assume your ratings matrix is already on wandb:

```
import jax
import jax.numpy as jnp
import numpy as np
import pandas as pd
import os, json, wandb, math

from jax import grad, jit
from jax import random
from jax.experimental import sparse

key = random.PRNGKey(0)

wandb.login()
run = wandb.init(
    # Set entity to specify your username or team name
    entity="wandb-un",
    # Set the project where this run will be logged
    project="jax-mf",
    # associate the runs to the right dataset
    config={
        "dataset": "MF-Dataset",
    }
)

# note that we assume the dataset is a ratings table stored in wandb
artifact = run.use_artifact('stored-dataset:latest')
ratings_artifact = artifact.download()
ratings_artifact_blob = json.load(
    open(
        os.path.join(
            ratings_artifact,
            'ratings.table.json'
        )
    )
)

ratings_artifact_blob.keys()
# ['_type', 'column_types', 'columns', 'data', 'ncols', 'nrows']

ratings = pd.DataFrame( # user_id, item_id, rating, unix_timestamp
```

```

        data=ratings_artifact_blob['data'],
        columns=ratings_artifact_blob['columns']
    )

def start_pipeline(df):
    return df.copy()

def column_as_type(df, column: str, cast_type):
    df[column] = df[column].astype(cast_type)
    return df

def rename_column_value(df, target_column, prior_val, post_val):
    df[target_column] = df[target_column].replace({prior_val: post_val})
    return df

def split_dataframe(df, holdout_fraction=0.1):
    """Splits a DataFrame into training and test sets.
    Args:
        df: a dataframe.
        holdout_fraction: fraction of dataframe rows to use in the test set.
    Returns:
        train: dataframe for training
        test: dataframe for testing
    """
    test = df.sample(frac=holdout_fraction, replace=False)
    train = df[~df.index.isin(test.index)]
    return train, test

all_rat = (ratings
    .pipe(start_pipeline)
    .pipe(column_as_type, column='user_id', cast_type=int)
    .pipe(column_as_type, column='item_id', cast_type=int)
)

def ratings_to_sparse_array(ratings_df, user_dim, item_dim):
    indices = (np.array(ratings_df['user_id']), np.array(ratings_df['item_id']))
    values = jnp.array(ratings_df['rating'])

    return {
        'indices': indices,
        'values': values,
        'shape': [user_dim, item_dim]
    }

```

```

def random_normal(pr_key, shape, mu=0, sigma=1, ):
    return (mu + sigma * random.normal(pr_key, shape=shape))

x = random_normal(
    pr_key = random.PRNGKey(1701),
    shape=(10000,),
    mu = 1.0,
    sigma = 3.0,
) # these hyperparameters are pretty meaningless

def sp_mse_loss(A, params):
    U, V = params['users'], params['items']
    rows, columns = A['indices']
    estimator = -(U @ V.T)[(rows, columns)]
    square_err = jax.tree_map(
        lambda x: x**2,
        A['values']+estimator
    )
    return jnp.mean(square_err)

omse_loss = jit(sp_mse_loss)

```

Note that we've had to implement our own loss function here. This is a relatively straightforward mean square error (MSE) loss, but it's taking advantage of the sparse nature of our matrix. You may notice in the code that we've converted the matrix to a sparse representation, so it's important that our loss function cannot only take advantage of that representation, but also be written to utilize the JAX device arrays and mapping/jitting.

IS THAT LOSS FUNCTION REALLY RIGHT?

If you're curious about this loss function that appeared like magic, we understand. While writing this book, we were extremely uncertain about what the best implementation of this loss function that leverages JAX would look like. There are actually many reasonable approaches to this kind of optimization. To that end, we wrote a public experiment to benchmark several approaches [on Colab](#).

Next, we need to build model objects to handle our MF state as we train. This code, while essentially mostly template code, will set us up well to

feed the model into a training loop in a relatively memory-efficient way. This model was trained on 100 million entries for a few thousand epochs on a MacBook Pro in less than a day:

```
class CFModel(object):
    """Simple class that represents a collaborative filtering model"""
    def __init__(
        self,
        metrics: dict,
        embeddings: dict,
        ground_truth: dict,
        embeddings_parameters: dict,
        prng_key=None
    ):
        """Initializes a CFModel.
        Args:
        """
        self._metrics = metrics
        self._embeddings = embeddings
        self._ground_truth = ground_truth
        self._embeddings_parameters = embeddings_parameters

        if prng_key is None:
            prng_key = random.PRNGKey(0)
        self._prng_key = prng_key

    @property
    def embeddings(self):
        """The embeddings dictionary."""
        return self._embeddings

    @embeddings.setter
    def embeddings(self, value):
        self._embeddings = value

    @property
    def metrics(self):
        """The metrics dictionary."""
        return self._metrics

    @property
    def ground_truth(self):
```

```

        """The train/test dictionary."""
        return self._ground_truth

def reset_embeddings(self):
    """Clear out embeddings state."""

    prng_key1, prng_key2 = random.split(self._prng_key, 2)

    self._embeddings['users'] = random_normal(
        prng_key1,
        [
            self._embeddings_parameters['user_dim'],
            self._embeddings_parameters['embedding_dim']
        ],
        mu=0,
        sigma=self._embeddings_parameters['init_stddev'],
    )
    self._embeddings['items'] = random_normal(
        prng_key2,
        [
            self._embeddings_parameters['item_dim'],
            self._embeddings_parameters['embedding_dim']]
        ,
        mu=0,
        sigma=self._embeddings_parameters['init_stddev'],
    )

def model_constructor(
    ratings_df,
    user_dim,
    item_dim,
    embedding_dim=3,
    init_stddev=1.,
    holdout_fraction=0.2,
    prng_key=None,
    train_set=None,
    test_set=None,
):
    if prng_key is None:
        prng_key = random.PRNGKey(0)

    prng_key1, prng_key2 = random.split(prng_key, 2)

    if (train_set is None) and (test_set is None):
        train, test = (ratings_df

```

```

        .pipe(start_pipeline)
        .pipe(split_dataframe, holdout_fraction=holdout_fraction)
    )

    A_train = (train
        .pipe(start_pipeline)
        .pipe(ratings_to_sparse_array, user_dim=user_dim, item_dim=item_dim)
    )
    A_test = (test
        .pipe(start_pipeline)
        .pipe(ratings_to_sparse_array, user_dim=user_dim, item_dim=item_dim)
    )
elif (train_set is None) ^ (test_set is None):
    raise('Must send train and test if sending one')
else:
    A_train, A_test = train_set, test_set

U = random_normal(
    prng_key1,
    [user_dim, embedding_dim],
    mu=0,
    sigma=init_stddev,
)
V = random_normal(
    prng_key2,
    [item_dim, embedding_dim],
    mu=0,
    sigma=init_stddev,
)

train_loss = omse_loss(A_train, {'users': U, 'items': V})
test_loss = omse_loss(A_test, {'users': U, 'items': V})

metrics = {
    'train_error': train_loss,
    'test_error': test_loss
}
embeddings = {'users': U, 'items': V}
ground_truth = {
    "A_train": A_train,
    "A_test": A_test
}
return CFModel(

```

```

        metrics=metrics,
        embeddings=embeddings,
        ground_truth=ground_truth,
        embeddings_parameters={
            'user_dim': user_dim,
            'item_dim': item_dim,
            'embedding_dim': embedding_dim,
            'init_stddev': init_stddev,
        },
        prng_key=prng_key,
    )

mf_model = model_constructor(all_rat, user_count, item_count)

```

We should also set this up to log nicely to wandb so it's easy to understand what is happening during training:

```

def train():
    run_config = { # These will be hyperparameters we will tune via wandb
        'emb_dim': 10, # Latent dimension
        'prior_std': 0.1, # Std dev around 0 for weights initialization
        'alpha': 1.0, # Learning rate
        'steps': 1500, # Number of training steps
    }

    with wandb.init() as run:
        run_config.update(run.config)
        model_object = model_constructor(
            ratings_df=all_rat,
            user_dim=user_count,
            item_dim=item_count,
            embedding_dim=run_config['emb_dim'],
            init_stddev=run_config['prior_std'],
            prng_key=random.PRNGKey(0),
            train_set=mf_model.ground_truth['A_train'],
            test_set=mf_model.ground_truth['A_test']
        )
        model_object.reset_embeddings() # Ensure we are starting from priors
        alpha, steps = run_config['alpha'], run_config['steps']
        print(run_config)
        grad_fn = jax.value_and_grad(omse_loss, 1)
        for i in range(steps):
            # We perform one gradient update

```

```

    loss_val, grads = grad_fn(
        model_object.ground_truth['A_train'],
        model_object.embeddings
    )
    model_object.embeddings = jax.tree_multimap(
        lambda p, g: p - alpha * g,
        # Basic update rule; JAX handles broadcasting for us
        model_object.embeddings,
        grads
    )
    if i % 1000 == 0: # Most output in wandb; little bit of logging
        print(f'Loss step {i}: ', loss_val)
        print(f""""Test loss: {
            omse_loss(
                model_object.ground_truth['A_train'],
                model_object.embeddings
            )}""")

    wandb.log({
        "Train omse": loss_val,
        "Test omse": omse_loss(
            model_object.ground_truth['A_test'],
            model_object.embeddings
        )
    })
})

```

Note that this code is using `tree_multimap` to handle broadcasting our update rule, and we're using the jitted loss from before in the `omse_loss` call. Also, we're calling `value_and_grad` so we can log the loss to wandb as we go. This is a common trick you'll see for efficiently doing both without a callback.

You can finish this off and start the training with a sweep:

```

sweep_config = {
    "name" : "mf-test-sweep",
    "method" : "random",
    "parameters" : {
        "steps" : {
            "min": 1000,
            "max": 3000,
        },
    },
}

```

```

        "alpha" :{
            "min": 0.6,
            "max": 1.75
        },
        "emb_dim" :{
            "min": 3,
            "max": 10
        },
        "prior_std" :{
            "min": .5,
            "max": 2.0
        },
    },
    "metric" : {
        'name': 'Test omse',
        'goal': 'minimize'
    }
}

sweep_id = wandb.sweep(sweep_config, project="jax-mf", entity="wandb-un")

wandb.init()
train()

count = 50
wandb.agent(sweep_id, function=train, count=count)

```

In this case, the hyperparameter optimization (HPO) is over our hyperparameters like embedding dimension and the priors (randomized matrices). Up until now, we have trained some MF models on our ratings matrix. Let's now add regularization and cross-validation.

Let's translate the preceding math equations directly into code:

```

def ell_two_regularization_term(params, dimensions):
    U, V = params['users'], params['items']
    N, M = dimensions['users'], dimensions['items']
    user_sq = jnp.multiply(U, U)
    item_sq = jnp.multiply(V, V)
    return (jnp.sum(user_sq)/N + jnp.sum(item_sq)/M)

l2_loss = jit(ell_two_regularization_term)

```

```

def gramian_regularization_term(params, dimensions):
    U, V = params['users'], params['items']
    N, M = dimensions['users'], dimensions['items']
    gr_user = U.T @ U
    gr_item = V.T @ V
    gr_square = jnp.multiply(gr_user, gr_item)
    return (jnp.sum(gr_square)/(N*M))

gr_loss = jit(gramian_regularization_term)

def regularized_omse(A, params, dimensions, hyperparams):
    lr, lg = hyperparams['ell_2'], hyperparams['gram']
    losses = {
        'omse': sp_mse_loss(A, params),
        'l2_loss': l2_loss(params, dimensions),
        'gr_loss': gr_loss(params, dimensions),
    }
    losses.update({
        'total_loss': losses['omse'] + lr*losses['l2_loss'] + lg*losses['gr_loss']
    })
    return losses['total_loss'], losses

reg_loss_observed = jit(regularized_omse)

```

We won't dive super deep into learning rate schedulers, but we will do a simple decay:

```

def lr_decay(
    step_num,
    base_learning_rate,
    decay_pct = 0.5,
    period_length = 100.0
):
    return base_learning_rate * math.pow(
        decay_pct,
        math.floor((1+step_num)/period_length)
    )

```

Our updated train function will incorporate our new regularizations—which come with some hyperparameters—and a bit of additional logging

setup. This code makes it easy to log our experiment as it trains and configures the hyperparameters to work with regularization:

```
def train_with_reg_loss():
    run_config = { # These will be hyperparameters we will tune via wandb
        'emb_dim': None,
        'prior_std': None,
        'alpha': None, # Learning rate
        'steps': None,
        'ell_2': 1, #l2 regularization penalization weight
        'gram': 1, #gramian regularization penalization weight
        'decay_pct': 0.5,
        'period_length': 100.0
    }

    with wandb.init() as run:
        run_config.update(run.config)
        model_object = model_constructor(
            ratings_df=all_rat,
            user_dim=942,
            item_dim=1681,
            embedding_dim=run_config['emb_dim'],
            init_stddev=run_config['prior_std'],
            prng_key=random.PRNGKey(0),
            train_set=mf_model.ground_truth['A_train'],
            test_set=mf_model.ground_truth['A_test']
        )
        model_object.reset_embeddings() # Ensure we start from priors

        alpha, steps = run_config['alpha'], run_config['steps']
        print(run_config)

        grad_fn = jax.value_and_grad(
            reg_loss_observed,
            1,
            has_aux=True
        ) # Tell JAX to expect an aux dict as output

        for i in range(steps):
            (total_loss_val, loss_dict), grads = grad_fn(
                model_object.ground_truth['A_train'],
                model_object.embeddings,
                dimensions={'users': user_count, 'items': item_count},
```



```

hyperparams={
    'ell_2': run_config['ell_2'],
    'gram': run_config['gram']
} # JAX carries our loss dict along for logging
)

model_object.embeddings = jax.tree_multimap(
    lambda p, g: p - lr_decay(
        i,
        alpha,
        run_config['decay_pct'],
        run_config['period_length']
    ) * g, # update with decay
    model_object.embeddings,
    grads
)
if i % 1000 == 0:
    print(f'Loss step {i}:')
    print(loss_dict)
    print(f"""Test loss: {
        omse_loss(model_object.ground_truth['A_test'],
        model_object.embeddings)}""")

loss_dict.update( # wandb takes the entire loss dictionary
    {
        "Test omse": omse_loss(
            model_object.ground_truth['A_test'],
            model_object.embeddings
        ),
        "learning_rate": lr_decay(i, alpha),
    }
)
wandb.log(loss_dict)

sweep_config = {
    "name" : "mf-HPO-with-reg",
    "method" : "random",
    "parameters" : {
        "steps": {
            "value": 2000
        },
        "alpha" :{
            "min": 0.6,

```

```

        "max": 2.25
    },
    "emb_dim" :{
        "min": 15,
        "max": 80
    },
    "prior_std" :{
        "min": .5,
        "max": 2.0
    },
    "ell_2" :{
        "min": .05,
        "max": 0.5
    },
    "gram" :{
        "min": .1,
        "max": .75
    },
    "decay_pct" :{
        "min": .2,
        "max": .8
    },
    "period_length" :{
        "min": 50,
        "max": 500
    }
},
"metric" : {
    'name': 'Test omse',
    'goal': 'minimize'
}
}

```

```

sweep_id = wandb.sweep(
    sweep_config,
    project="jax-mf",
    entity="wandb-un"
)

```

```

run_config = { # These will be hyperparameters we will tune via wandb
    'emb_dim': 10, # Latent dimension
    'prior_std': 0.1,
    'alpha': 1.0, # Learning rate
    'steps': 1000, # Number of training steps

```

```

        'ell_2': 1, #l2 regularization penalization weight
        'gram': 1, #gramian regularization penalization weight
        'decay_pct': 0.5,
        'period_length': 100.0
    }

    train_with_reg_loss()

```

The last step is to do this in a way that gives us confidence in the models we're seeing. Unfortunately, setting up cross-validation for MF problems can be tricky, so we'll need to make a few modifications to our data structures:

```

def sparse_array_concatenate(sparse_array_iterable):
    return {
        'indices': tuple(
            map(
                jnp.concatenate,
                zip(*(x['indices'] for x in sparse_array_iterable)))
        ),
        'values': jnp.concatenate(
            [x['values'] for x in sparse_array_iterable]
        ),
    }

class jax_df_Kfold(object):
    """Simple class that handles Kfold
    splitting of a matrix as a dataframe and stores as sparse jarrays"""
    def __init__(
        self,
        df: pd.DataFrame,
        user_dim: int,
        item_dim: int,
        k: int = 5,
        prng_key=random.PRNGKey(0)
    ):
        self._df = df
        self._num_folds = k
        self._split_idxes = jnp.array_split(
            random.permutation(
                prng_key,
                df.index.to_numpy(),

```

```

        axis=0,
        independent=True
    ),
    self._num_folds
)

self._fold_arrays = dict()

for fold_index in range(self._num_folds):
    # let's create sparse jax arrays for each fold piece
    self._fold_arrays[fold_index] = (
        self._df[
            self._df.index.isin(self._split_idxes[fold_index])
        ].pipe(start_pipeline)
        .pipe(
            ratings_to_sparse_array,
            user_dim=user_dim,
            item_dim=item_dim
        )
    )

def get_fold(self, fold_index: int):
    assert(self._num_folds > fold_index)
    test = self._fold_arrays[fold_index]
    train = sparse_array_concatenate(
        [v for k,v in self._fold_arrays.items() if k != fold_index]
    )
    return train, test

```

Each hyperparameter setup should yield loss for each fold, so within *wandb.init*, we build a model with each fold:

```

for j in num_folds:
    train, test = folder.get_fold(j)
    model_object_dict[j] = model_constructor(
        ratings_df=all_rat,
        user_dim=user_count,
        item_dim=item_count,
        embedding_dim=run_config['emb_dim'],
        init_stddev=run_config['prior_std'],
        prng_key=random.PRNGKey(0),
        train_set=train,

```

```

        test_set=test
    )

```

At each step, we'd like to not only compute the gradient for the training and evaluate on the test but also compute gradients for all folds, evaluate on all the tests, and produce the relevant errors:

```

for i in range(steps):
    loss_dict = {"learning_rate": step_decay(i)}
    for j, M in model_object_dict.items():
        (total_loss_val, fold_loss_dict), grads = grad_fn(
            M.ground_truth['A_train'],
            M.embeddings,
            dimensions={'users': 942, 'items': 1681},
            hyperparams={'ell_2': run_config['ell_2'], 'gram': run_config['gram']})

        M.embeddings = jax.tree_multimap(
            lambda p, g: p - step_decay(i) * g,
            M.embeddings,
            grads
        )

```

Logging should be losses per fold, and the aggregate loss should be the target metric. This is because each fold is an independent optimization of the model parameters; however, we wish to see aggregate behavior across the folds:

```

        fold_loss_dict = {f'{k}_fold-{j}': v for k, v in fold_loss_dict.items()}
        fold_loss_dict.update(
            {
                f"Test omse_fold-{j}": omse_loss(
                    M.ground_truth['A_test'],
                    M.embeddings
                ),
            }
        )

        loss_dict.update(fold_loss_dict)

    loss_dict.update({

```

```

        "Test omse_mean": jnp.mean(
            [v for k,v in loss_dict.items() if k.startswith('Test omse_fold-')]
        )
    })
    wandb.log(loss_dict)

```

We wrap up into one big training method:

```

def train_with_reg_loss_CV():
    run_config = { # These will be hyperparameters we will tune via wandb
        'emb_dim': None, # Latent dimension
        'prior_std': None,
        # Standard deviation around 0 that our weights are initialized to
        'alpha': None, # Learning rate
        'steps': None, # Number of training steps
        'num_folds': None, # Number of CV Folds
        'ell_2': 1, #hyperparameter for l2 regularization penalization weight
        'gram': 1, #hyperparameter for gramian regularization penalization weigh
    }

    with wandb.init() as run:
        run_config.update(run.config) # This is how the wandb agent passes param
        model_object_dict = dict()

        for j in range(run_config['num_folds']):
            train, test = folder.get_fold(j)
            model_object_dict[j] = model_constructor(
                ratings_df=all_rat,
                user_dim=942,
                item_dim=1681,
                embedding_dim=run_config['emb_dim'],
                init_stddev=run_config['prior_std'],
                prng_key=random.PRNGKey(0),
                train_set=train,
                test_set=test
            )
            model_object_dict[j].reset_embeddings()
            # Ensure we are starting from priors

        alpha, steps = run_config['alpha'], run_config['steps']
        print(run_config)

        grad_fn = jax.value_and_grad(reg_loss_observed, 1, has_aux=True)

```

```

# Tell JAX to expect an aux dict as output

for i in range(steps):
    loss_dict = {
        "learning_rate": lr_decay(
            i,
            alpha,
            decay_pct=.75,
            period_length=250
        )
    }
    for j, M in model_object_dict.items():
        # Iterate through folds

        (total_loss_val, fold_loss_dict), grads = grad_fn(
            # compute gradients for one fold
            M.ground_truth['A_train'],
            M.embeddings,
            dimensions={'users': 942, 'items': 1681},
            hyperparams={
                'ell_2': run_config['ell_2'],
                'gram': run_config['gram']
            }
        )

        M.embeddings = jax.tree_multimap(
            # update weights for one fold
            lambda p, g: p - lr_decay(
                i,
                alpha,
                decay_pct=.75,
                period_length=250
            ) * g,
            M.embeddings,
            grads
        )

        fold_loss_dict = {
            f'{k}_fold-{j}':
                v for k, v in fold_loss_dict.items()
        }
        fold_loss_dict.update( # loss calculation within fold
            {
                f"Test omse_fold-{j}": omse_loss(

```

```

        M.ground_truth['A_test'],
        M.embeddings
    ),
    }
)

loss_dict.update(fold_loss_dict)

loss_dict.update({ # average loss over all folds
    "Test omse_mean": np.mean(
        [v for k,v in loss_dict.items()
         if k.startswith('Test omse_fold-')]
    ),
    "test omse_max": np.max(
        [v for k,v in loss_dict.items()
         if k.startswith('Test omse_fold-')]
    ),
    "test omse_min": np.min(
        [v for k,v in loss_dict.items()
         if k.startswith('Test omse_fold-')]
    )
})
wandb.log(loss_dict)

if i % 1000 == 0:
    print(f'Loss step {i}:')
    print(loss_dict)

```

Here's our final sweeps configuration:

```

sweep_config = {
    "name" : "mf-HPO-CV",
    "method" : "random",
    "parameters" : {
        "steps": {
            "value": 2000
        },
        "num_folds": {
            "value": 5
        },
        "alpha" :{
            "min": 2.0,
            "max": 3.0
        }
    }
}

```



```

    },
    "emb_dim" :{
        "min": 15,
        "max": 70
    },
    "prior_std" :{
        "min": .75,
        "max": 1.0
    },
    "ell_2" :{
        "min": .05,
        "max": 0.5
    },
    "gram" :{
        "min": .1,
        "max": .6
    },
    },
    "metric" : {
        'name': 'Test omse_mean',
        'goal': 'minimize'
    }
}

sweep_id = wandb.sweep(sweep_config, project="jax-mf", entity="wandb-un")

wandb.agent(sweep_id, function=train_with_reg_loss_CV, count=count)

```

That may seem like a lot of setup, but we’ve really achieved a lot here. We’ve initialized the model to optimize the two matrix factors while simultaneously keeping the matrix elements and the Gramians small.

This brings us to our lovely images.

Output from HPO MF

Let’s have a quick look at what the prior work has produced. First, [Figure 10-3](#) shows that our primary loss function, observed mean square error (OMSE), is rapidly decreasing. This is great, but we should take a deeper look.

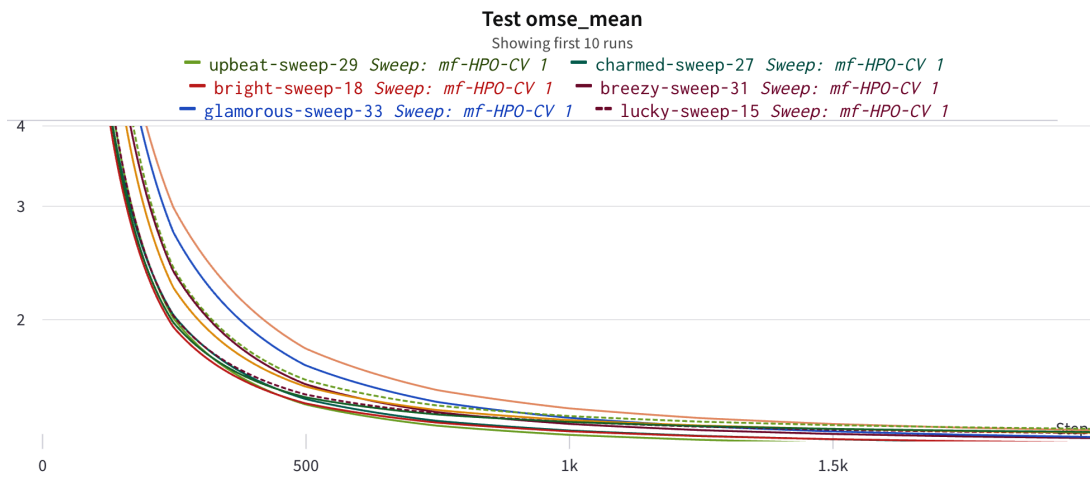


Figure 10-3. The loss during training

Let's also have a quick look to ensure that our regularization parameters (Figure 10-4) are converging. We can see that our L2 regularization could probably still decrease if we were to continue for more epochs.

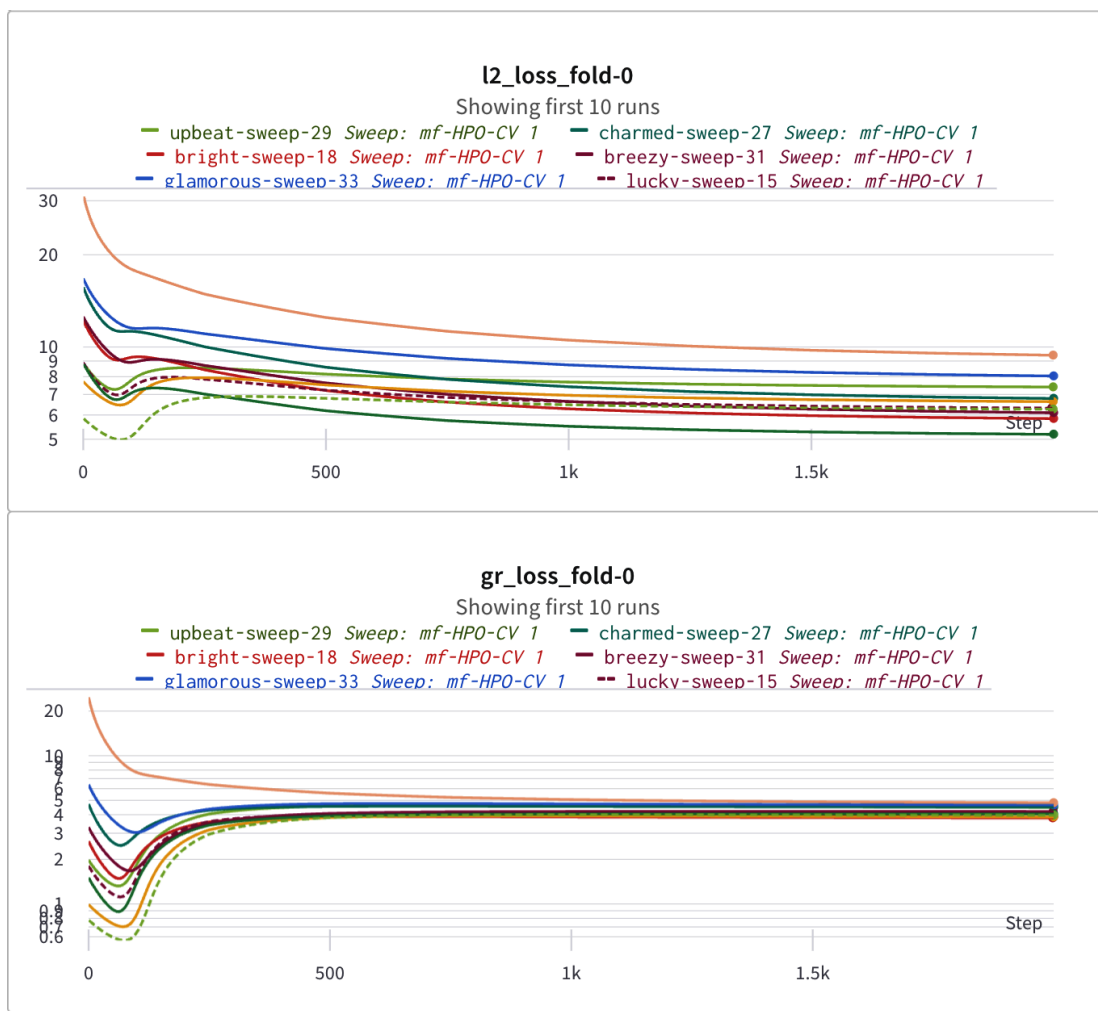


Figure 10-4. Regularization parameters

We'd like to see our cross-validation laid out by fold and corresponding loss (Figure 10-5). This is a *parallel coordinates chart*; its lines correspond

to different runs that are in correspondence with different choices of parameters, and its vertical axes are different metrics. The far-right heatmap axis corresponds to the overall total loss that we're trying to minimize. In this case, we alternate test loss on a fold and total loss on that fold. Lower numbers are better, and we hope to see individual lines consistent across their loss per fold (otherwise, we may have a skewed dataset). We see that choices of hyperparameters can interact with fold behavior, but in all the low-loss scenarios (at the bottom), we see a high correlation between performance on different folds (the vertical axes in the plot).

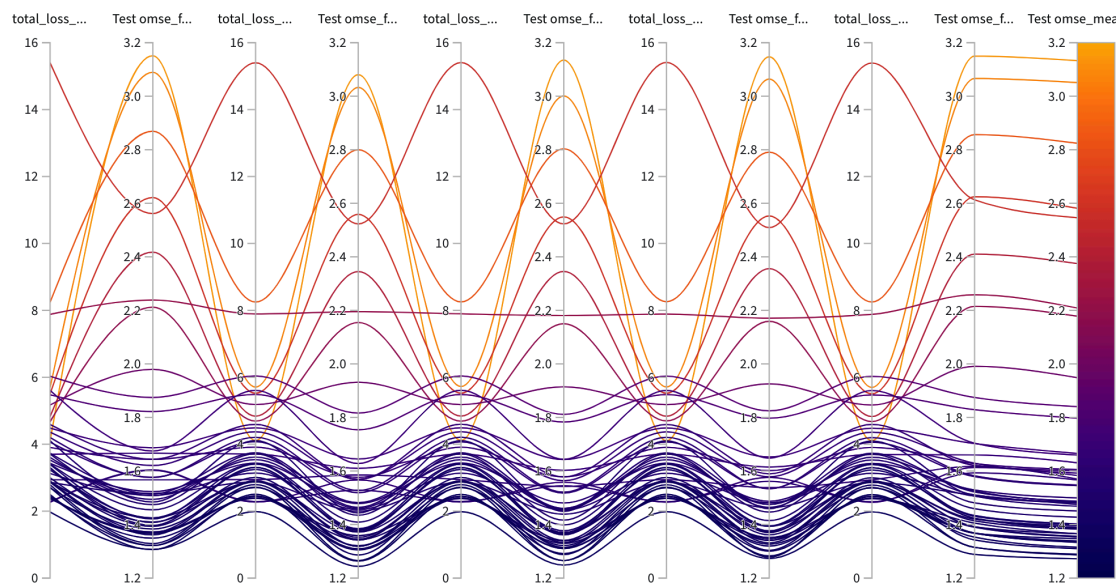


Figure 10-5. The loss during training

Next up, which choices of hyperparameters have a strong effect on performance? [Figure 10-6](#) is another parallel coordinates plot with the vertical axes corresponding to different hyperparameters. Generally, we're looking for which domains on the vertical axes correspond to low loss on the far-right heatmap. We see that some of our hyperparameters like priors distribution and, somewhat surprisingly, `e11_2` have virtually no effect. However, small embedding dimension and small Gramian weight definitely do. A larger alpha also seems to correlate well with good performance.

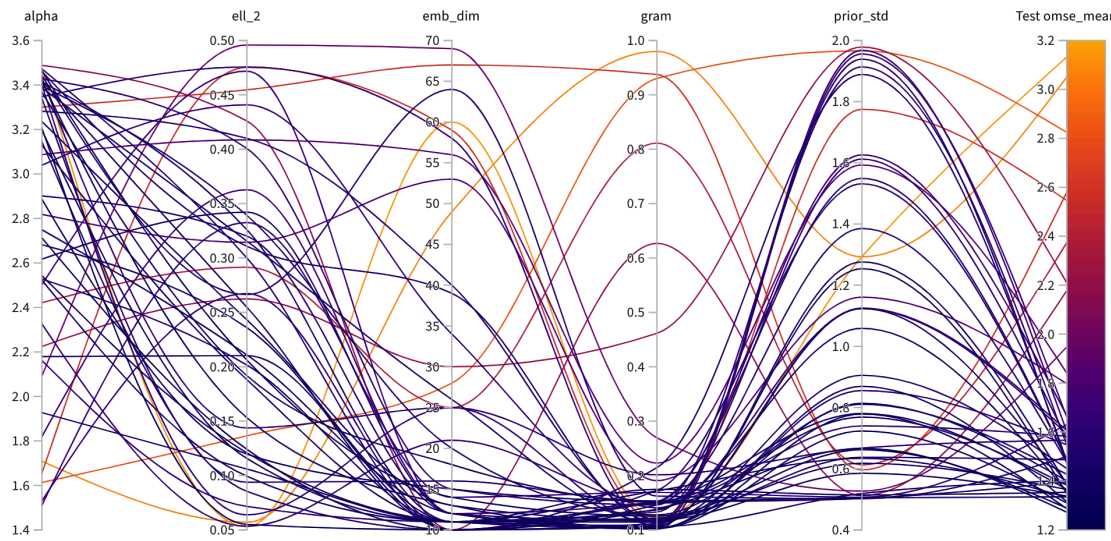


Figure 10-6. The loss by hyperparameter

Finally, we see that as we do a Bayesian hyperparameter search, we really do improve our performance over time. [Figure 10-7](#) is a Pareto plot in which each dot in the scatterplot represents one run, and left to right is a time axis. The vertical axis is overall total loss, so lower is better, and it means that generally we're converging toward better performance. The line inscribed along the bottom of the convex hull of the scatter points is the *Pareto frontier*, or the best performance at that x value. Since this is a time-series Pareto plot, it merely tracks the best performance in time.

You may be wondering how and why we're able to converge to better loss values in time. This is because we've conducted a Bayesian hyperparameter search, which means we selected our hyperparameters from independent Gaussians, and we updated our priors for each parameter based on performance of previous runs. For an introduction to this method, see [“Bayesian Hyperparameter Optimization—A Primer”](#) by Robert Mitson. In a real setting, we'd see less monotonic behavior in this plot, but we'd always be hoping to improve.

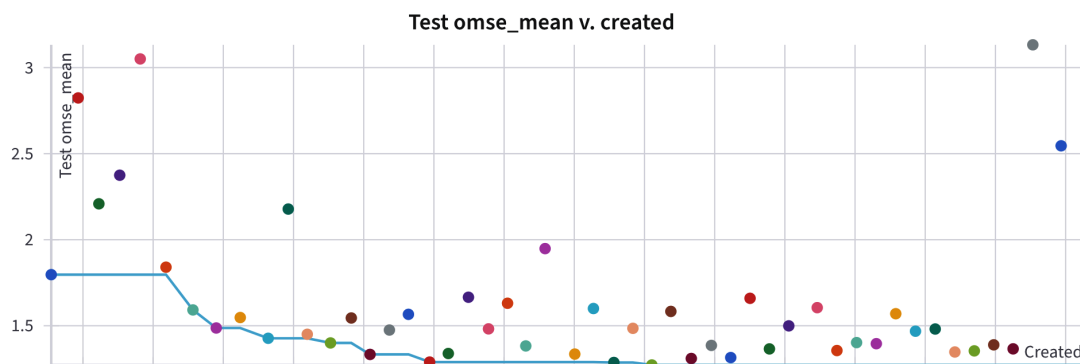


Figure 10-7. The Pareto frontier of the loss values

Prequential validation

If we were to put the preceding approach into practice, we would need to capture our trained models in a model registry for use in production. Best practice is to establish a set of explicit evaluations against which to test a selection of models. In your basic ML training, you've likely been encouraged to think about validation datasets; these may take many forms, testing particular subsets of instances or features or even distributed across covariates in a known way.

One useful framing for recommendation systems is to remember that they're a fundamentally sequential dataset. With this in mind, let's take another look at our ratings data. Later we will talk more about sequential recommenders, but while we're talking about validation, it's useful to mention how to take proper care.

Notice that all our ratings have an associated timestamp. To build a proper validation set, it's a good idea to take that timestamp from the end of our data.

However, you might be wondering, "When are different users active?" and "Is it possible that the later timestamps are a biased selection of the ratings?" These are important questions. To account for these questions, we should do a holdout by user.

To create this *prequential dataset*, where the test set follows directly after the training set in a chronological sequence, start by deciding on a desired size for validation, like 10%. Next, group the data by user. Finally, employ rejection sampling, ensuring you don't use the most recent timestamp as the rejection criterion.

Here's a simple implementation for pandas using rejection sampling. This is not the most computationally efficient implementation, but it will get the job done:

```
def prequential_validation_set(df, holdout_perc=0.1):  
    ...  
    We utilize rejection sampling.
```

Assign a probability to all observations, if they lie below the sample percentage AND they're the most recent still in the set, include.

Otherwise return them and repeat.

Each time, take no more than the remaining necessary to fill the count.
'''

```
count = int(len(df)*holdout_perc)
sample = []
while count > 0:
    df['p'] = np.random.rand(len(df),1) #generate probabilities
    x = list(
        df.loc[~df.index.isin(sample)] # exclude already selected
        .sort_values(['unix_timestamp'], ascending=False)
        .groupby('user_id').head(1) # only allow the first in each group
        .query("p < @holdout_perc").index # grab the indices
    )
    rnd.shuffle(x) # ensure our previous sorting doesn't bias the users subset
    sample += x[:count] # add observations up to the remaining needed
    count -= len(x[:count]) # decrement the remaining needed

df.drop(columns=['p'], inplace=True)

test = df.iloc[sample]
train = df[~df.index.isin(test.index)]
return train, test
```

This is an effective and important validation scheme for inherently sequential datasets.

WSABIE

Let's focus again on optimizations and modifications. Another optimization is to treat the MF problem as a single optimization.

The paper [“WSABIE: Scaling Up to Large Vocabulary Image Annotation”](#) by Jason Weston et al. also contains a factorization for just the item matrix. In this scheme, we replace the user matrix with a weighted sum of the items a user has affinity to. We cover web scale annotation by image embedding (WSABIE) and Warp loss in [“WARP”](#). Representing a user as the average of items they like is a way of saving space and not needing a separate user matrix if there are large numbers of users.

A completely alternative way to do HPO for RecSys is via the latent spaces themselves! [“Hyper-Parameter Optimization for Latent Spaces in Dynamic Recommender Systems”](#) by Bruno Veloso et al. attempts to modify the relative embeddings during each step to optimize the embedding model.

Dimension Reduction

Dimension-reduction techniques are frequently employed in recommendation systems to decrease computational complexity and enhance the accuracy of recommendation algorithms. In this context, the primary concepts of dimension reduction for recommendation systems include MF and SVD.

The *matrix factorization method* decomposes the user-item interaction matrix ($A \in \mathbb{R}^{(m \times n)}$) into two lower-dimensional matrices, representing the user ($U \in \mathbb{R}^{(m \times r)}$) and item ($V \in \mathbb{R}^{(n \times r)}$) latent factors, respectively. This technique can reveal the underlying data structure and offer recommendations based on a user’s previous interactions.

Mathematically, MF can be represented as follows:

$$A \sim U \times V^{(T)}$$

SVD is a linear-algebra technique that decomposes a matrix (A) into three matrices—the left singular vectors (U), the singular values (Σ), and the right singular vectors (V). SVD can be utilized for MF in recommendation systems, where the user-item interaction matrix is decomposed into a smaller number of latent factors. The mathematical representation of SVD is as follows:

$$A = U \times \Sigma \times V^{(T)}$$

In practice, though, rather than using a mathematical library to find the eigenvectors, folks might use the [power iteration method](#) to discover the eigenvectors approximately. This method is far more scalable than a full dense SVD solution that is optimized for correctness and dense vectors:


```

import jax
import jax.numpy as jnp

def power_iteration(a: jnp.ndarray) -> jnp.ndarray:
    """Returns an eigenvector of the matrix a.
    Args:
        a: a n x m matrix
    """
    key = jax.random.PRNGKey(0)
    x = jax.random.normal(key, shape=(a.shape[1], 1))
    for i in range(100):
        x = a @ x
        x = x / jnp.linalg.norm(x)
    return x.T

key = jax.random.PRNGKey(123)
A = jax.random.normal(key, shape=[4, 4])
print(A)
[[ 0.52830553  0.3722206 -1.2219944 -0.10314374]
 [ 1.4722222  0.47889313 -1.2940298  1.0449569 ]
 [ 0.23724185  0.3545859 -0.172465 -1.8011322 ]
 [ 0.4864215  0.08039388 -1.2540827  0.72071517]]
S, _, _ = jnp.linalg.svd(A)
print(S)
[[ -0.375782   0.40269807  0.44086716 -0.70870167]
 [ -0.753597   0.0482972 -0.65527284  0.01940039]
 [  0.2040088  0.91405433 -0.15798494  0.31293103]
 [ -0.49925917 -0.00250015  0.5927009  0.6320123 ]]
x1 = power_iteration(A)
print(x1.T)
[[ -0.35423845]
 [ -0.8332922 ]
 [  0.16189891]
 [ -0.39233655]]

```

Notice that the eigenvector returned by the power iteration is close to the first column of S , but not quite. This is because the method is approximate. It relies on the fact that an eigenvector doesn't change in direction when multiplied by the matrix. So by repeatedly multiplying by the matrix, we eventually iterate onto an eigenvector. Also notice that we solved for column eigenvectors instead of the row eigenvectors. In this example,

the columns are users, and the rows are items. It is important to play with transposed matrices because a lot of ML involves reshaping and transposing matrices, so getting used to them early is an important skill.

EIGENVECTOR EXAMPLES

Here's a nice exercise for you: the second eigenvector is computed by subtracting out the first eigenvector after the matrix multiplication. This is telling the algorithm to ignore any component along the first eigenvector in order to compute the second eigenvector. As a fun exercise, hop over to [Colab](#) and try computing the second eigenvector. Extending this to sparse vector representations is another interesting exercise, as it allows you to start computing the eigenvectors of sparse matrices, which is usually the form of matrix that recommender systems use.

Next, we construct a recommendation for a user by creating a column and then taking the dot product with all the eigenvectors and finding the closest. We then find all the highest-scoring entries in the eigenvector that the user hasn't seen and return them as recommendations. So in the preceding example, if the eigenvector $\mathbf{x}_1$ was the closest to the user column, then the best item to recommend would be item 3 because it is the largest component in the eigenvector and thus rated most highly if the user is closest to the eigenvector $\mathbf{x}_1$. Here's what this looks like in code:

```
import jax
import jax.numpy as jnp

def recommend_items(eigenvectors: jnp.ndarray, user:jnp.ndarray) -> jnp.ndarray:
    """Returns an ordered list of recommend items for the user.
    Args:
        eigenvectors: a nxm eigenvector matrix
        user: a user vector of size m.
    """
    score_eigenvectors = jnp.matmul(eigenvectors.T, user)
    which_eigenvector = jnp.argmax(score_eigenvectors)
    closest_eigenvector = eigenvectors.T[which_eigenvector]
    scores, items = jax.lax.top_k(closest_eigenvector, 3)
    return scores, items

S = jnp.array(
    [[-0.375782,    0.40269807],
```

```

[-0.753597,  0.0482972],
[ 0.2040088,  0.91405433],
[-0.49925917, -0.00250015]])
u = jnp.array([-1, -1, 0, 0]).reshape(4, 1)
scores, items = recommend_items(S, u)
print(scores)
[ 0.2040088 -0.375782 -0.49925917]
print(items)
[2 0 3]

```

In this example, a user has downvoted item 0 and item 1. The closest column eigenvector is therefore column 0. We then select the closest eigenvector to the user, order the entries, and recommend item 2 to the user, which is the highest-scoring entry that the user has not seen.

Two techniques aim to extract the most relevant features from the user-item interaction matrix and reduce its dimensionality, which can improve performance:

Principal component analysis (PCA)

This statistical technique transforms the original high-dimensional data into a lower-dimensional representation while retaining the most important information. PCA can be applied to the user-item interaction matrix to reduce the number of dimensions and improve the computational efficiency of the recommendation algorithm.

Nonnegative matrix factorization (NMF)

This technique decomposes the nonnegative user-item interaction matrix ($A \in \mathbb{R}^{(m \times n)^{*+}}$) into two nonnegative matrices ($W \in \mathbb{R}^{(m \times r)^{*+}}$ and $H \in \mathbb{R}^{(r \times n)^{+}}$). NMF can be utilized for dimension reduction in recommendation systems, where the latent factors are nonnegative and interpretable. The mathematical representation of NMF is $A \simeq W \times H$.

MF techniques can be further extended to incorporate additional information, such as item content or user demographic data, through the use of side information. Side information can be employed to augment the

user-item interaction matrix, allowing for more accurate and personalized recommendations.

Furthermore, MF models can be extended to handle implicit feedback data, where the absence of interaction data is not equivalent to the lack of interest. By incorporating additional regularization terms into the objective function, MF models can learn a more robust representation of the user-item interaction matrix, leading to better recommendations for implicit feedback scenarios.

Consider a recommendation system that employs MF to model the user-item interaction matrix. If the system comprises many users and items, the resulting factor matrices can be high-dimensional and computationally expensive to process. However, by using dimension-reduction techniques like SVD or PCA, the algorithm can reduce the dimensionality of the factor matrices while preserving the most important information about the user-item interactions. This enables the algorithm to generate more efficient and accurate recommendations, even for new users or items with limited interaction data.

Isometric Embeddings

Isometric embeddings are a specific type of embedding that maintains distances between points in high-dimensional space when mapping them onto a lower-dimensional space. The term *isometric* signifies that the distances between points in the high-dimensional space are preserved precisely in the lower-dimensional space, up to a scaling factor.

In contrast to other types of embeddings, such as linear or nonlinear embeddings, which may distort the distances between points, isometric embeddings are preferable in numerous applications where distance preservation is essential. For example, in ML, isometric embeddings can be employed to visualize high-dimensional data in two or three dimensions while preserving the relative distances between the data points. In NLP, isometric embeddings can be utilized to represent the semantic similarities between words or documents while maintaining their relative distances in the embedding space.

One popular technique for generating isometric embeddings is *multidimensional scaling* (MDS). MDS operates by computing pairwise distances between the data points in the high-dimensional space and then determining a lower-dimensional embedding that preserves these distances. The optimization problem is generally formulated as a constrained optimization problem, where the objective is to minimize the difference between the pairwise distances in the high-dimensional space and the corresponding distances in the lower-dimensional embedding.

Mathematically, we write: $\min_{(X)} \sum_{(i,j)} (d_{ij} - \|x_i - x_j\|)^2$.

Here, d_{ij} denotes the pairwise distances in the high-dimensional space, and x_i and x_j represent points in the lower-dimensional embedding.

Another approach for generating isometric embeddings is through the use of kernel methods, such as kernel PCA or kernel MDS. Kernel methods work by implicitly mapping the data points into a higher-dimensional feature space, where the distances between the points are easier to compute. The isometric embedding is then calculated in the feature space, and the resulting embedding is mapped back to the original space.

Isometric embeddings have been employed in recommendation systems to represent the user-item interaction matrix in a lower-dimensional space where the distances between the items are preserved. By preserving the distances between items in the embedding space, the recommendation algorithm can better capture the underlying structure of the data and provide more accurate and diverse recommendations.

Isometric embeddings can also be employed to incorporate additional information into the recommendation algorithm, such as item content or user demographic data. By using isometric embeddings to represent the items and the additional information, the algorithm can capture the similarities between items based on both the user-item interaction data and the item content or user demographics, leading to more accurate and diverse recommendations.

Moreover, isometric embeddings can also be used to address the cold-start problem in recommendation systems. By using the isometric embeddings to represent the items, the algorithm can make recommendations

for new items based on their similarities to the existing items in the embedding space, even in the absence of user interactions.

In summary, isometric embeddings are a valuable technique in recommendation systems for representing the user-item interaction matrix in a lower-dimensional space where the distances between the items are preserved. Isometric embeddings can be generated using MF techniques and can be employed to incorporate additional information, address the cold-start problem, and improve the accuracy and diversity of recommendations.

Nonlinear Locally Metrizable Embeddings

Nonlinear locally metrizable embeddings are yet another method to represent the user-item interaction matrix in a lower-dimensional space where the local distances between nearby items are preserved. By preserving the local distances between items in the embedding space, the recommendation algorithm can better capture the local structure of the data and provide more accurate and diverse recommendations.

Mathematically, let $\mathbf{X} = \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ be the set of items in the high-dimensional space, and $\mathbf{Y} = \mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n$ be the set of items in the lower-dimensional space. The goal of nonlinear locally metrizable embeddings is to find a mapping $f : \mathbf{X} \rightarrow \mathbf{Y}$ that preserves the local distances, i.e., for any $\mathbf{x}_i, \mathbf{x}_j \in \mathbf{X}$, we have this:

$$d_Y(f(\mathbf{x}_i), f(\mathbf{x}_j)) \simeq d_X(\mathbf{x}_i, \mathbf{x}_j)$$

One popular approach to generating nonlinear locally metrizable embeddings in recommendation systems is via autoencoder neural networks. Autoencoders work by mapping the high-dimensional user-item interaction matrix onto a lower-dimensional space through an encoder network, and then reconstructing the matrix back in the high-dimensional space through a decoder network. The encoder and decoder networks are trained jointly to minimize the difference between the input data and the reconstructed data, with the objective of capturing the underlying structure of the data in the embedding space:

$$\min_{(\theta, \varphi)} \sum_{(i=1)}^n \|x_i - g_{\varphi}(f_{\theta}(x_i))\|^2$$

Here, f_{θ} denotes the encoder network with parameters θ , g_{θ} denotes the decoder network with parameters θ , and $\|\cdot\|$ represents the Euclidean norm.

Another approach for generating nonlinear locally metrizable embeddings in recommendation systems is through the use of t-distributed stochastic neighbor embedding (t-SNE). t-SNE works by modeling the pairwise similarities between the items in the high-dimensional space, and then finding a lower-dimensional embedding that preserves these similarities.

A more popular approach in modern times is UMAP, which instead attempts to fit a minimal manifold that preserves density in local neighborhoods. UMAP is an essential technique for finding low-dimensional representations in complex and high-dimensional latent spaces; find its documentation at <https://oreil.ly/NLqDg>. The optimization problem is typically formulated as a cost function C that measures the difference between the pairwise similarities in the high-dimensional space and the corresponding similarities in the lower-dimensional embedding:

$$C(Y) = \sum_{(i,j)} p_{ij} * \log \left(\frac{p_{ij}}{q_{ij}} \right)$$

Here, p_{ij} denotes the pairwise similarities in the high-dimensional space, q_{ij} denotes the pairwise similarities in the lower-dimensional space, and the sum is over all pairs of items (i, j) .

Nonlinear locally metrizable embeddings can also be used to incorporate additional information into the recommendation algorithm, such as item content or user demographic data. By using nonlinear locally metrizable embeddings to represent the items and the additional information, the algorithm can capture the similarities between items based on both the user-item interaction data and the item content or user demographics, leading to more accurate and diverse recommendations.

Moreover, nonlinear locally metrizable embeddings can also be used to address the cold-start problem in recommendation systems. By using the

nonlinear locally metrizable embeddings to represent the items, the algorithm can make recommendations for new items based on their similarities to the existing items in the embedding space, even in the absence of user interactions.

In summary, nonlinear locally metrizable embeddings are a useful technique in recommendation systems for representing the user-item interaction matrix in a lower-dimensional space where the local distances between nearby items are preserved. Nonlinear locally metrizable embeddings can be generated using techniques such as autoencoder neural networks or t-SNE and can be used to incorporate additional information, address the cold-start problem, and improve the accuracy and diversity of recommendations.

Centered Kernel Alignment

When training neural networks, the latent space representations at each layer are expected to express correlation structures between the incoming signals. Frequently, these interstitial representations comprise a sequence of states transitioning from the initial layer to the final layer. You may naturally wonder, “How do these representations change throughout the layers of the network” and “How similar are these layers?”

Interestingly, for some architectures, this question may yield deep insight into the network’s behavior.

This process of comparing layer representations is called *correlation analysis*. For an MLP with layers $1, \dots, N$, the correlations may be represented by an $N \times N$ matrix of pairwise relationships. The idea is that each layer comprises a series of latent factors, and similar to correlation analysis for other features of a dataset, these latent features’ relationships may be simply summarized by their covariance.

Affinity and p-sale

As you’ve seen, MF is a powerful dimension-reduction technique that can yield an estimator for the probability of a sale (often shorted to *p-sale*). In MF, the goal has been to decompose this historical data on user behavior

and the product sales matrix into two lower-dimensional matrices: one that represents user preferences and another that represents product characteristics. Now, let's convert this MF model into a sale estimator.

Let $R \in \mathbb{R}^{(M \times N)}$ be the historical data matrix, where M is the number of users and N is the number of products. The MF aims to find two matrices $U \in \mathbb{R}^{(M \times d)}$ and $V \in \mathbb{R}^{(N \times d)}$, where d is the dimensionality of the latent space, such that:

$$R \simeq U * V^T$$

The *probability of a sale*, or equivalently a read, watch, eat, or click, can be predicted using MF by first decomposing the historical data matrix into user and product matrices, and then calculating a score that represents the likelihood of a user purchasing a given product. This score can be calculated using the dot product of the corresponding row in the user matrix and the column in the product matrix, followed by a logistic function to transform the dot product into a probability score.

Mathematically, the probability of a sale for a user u and a product p can be represented as follows:

$$P(u, p) = \text{sigmoid}(u * p^T)$$

Here, sigmoid is the logistic function that maps the dot product of the user and product vectors to a probability score between 0 and 1:

$$\text{sigmoid}(x) = 1 / (1 + \exp(-x))$$

The p^T represents the transpose of the product vector. The dot product of the user and product vectors is a measure of the similarity between the user's preferences and the product's characteristics, and the logistic function maps this similarity score to a probability score.

The user and product matrices can be trained on the historical data by using various MF algorithms, such as SVD, NMF, or ALS. Once the matrices are trained, the dot product and logistic function can be applied to new user-product pairs to predict the probability of a sale. The predicted probabilities can then be used to rank and recommend products to the user.

It's worth highlighting that, since the loss function for ALS is convex (meaning there is a single global minimum), the convergence can be fast when we fix either the user or item matrix. In this method, the user matrix is fixed and the item matrix is solved for. Then the item matrix is fixed and the user matrix is solved for. The method alternates between the two solutions, and because the loss is convex in this regime, the method converges quickly.

The dot product of the corresponding row in the user matrix and column in the product matrix represents the affinity score between the user and the product, or how well the user's preferences match the product's characteristics. However, this score alone may not be a sufficient predictor of whether the user will actually purchase the product.

The logistic function applied to the dot product in the MF model transforms the affinity score into a probability score, which represents the likelihood of a sale. This transformation takes into account additional factors beyond just the user's preferences and the product's characteristics, such as the overall popularity of the product, the user's purchasing behavior, and any other relevant external factors. By incorporating these additional factors, MF is able to better predict the probability of a sale, rather than just an affinity score.

A comparison library (however, not in JAX) for computing latent embeddings linearly is [libFM](#). The formulation for a factorization machine is similar to a GloVe embedding in that it also models the interaction between two vectors, but the dot product can be used for regression or binary classification tasks. The method can also be extended to recommend more than two kinds of items beyond user and item.

In summary, MF produces probabilities of sale instead of just affinity scores by incorporating additional factors beyond the user's preferences and the product's characteristics, and transforming the affinity score into a probability score by using a logistic function.

Propensity Weighting for

Recommendation System Evaluation

As you've seen, recommendation systems are evaluated based on user feedback, which is collected from the deployed recommendation system. However, this data is causally influenced by the deployed system, creating a feedback loop that may bias the evaluation of new models. This feedback loop can lead to confounding variables, making it difficult to distinguish between user preferences and the influence of the deployed system.

If this surprises you, let's consider for a moment what would have to be true for a recommendation system to *not* causally influence the actions users take and/or the outcomes that result from those actions. That would require assumptions like “the recommendations are completely ignored by the user” and “the system makes recommendations at random.” Propensity weighting can mitigate some of the worst effects of this problem.

The performance of a recommender system depends on many factors, including user-item characteristics, contextual information, and trends, which can affect the quality of the recommendations and the user engagement. However, the influence can be mutual: the user interactions influence the recommender, and vice versa. Evaluating the causal effect of a recommender system on user behavior and satisfaction is therefore a challenging task, as it requires controlling for potential confounding factors—those that may affect both the treatment assignment (the recommendation strategy) and the outcome of interest (the user's response to the recommendations).

Causal inference provides a framework for addressing these challenges. In the context of recommender systems, causal inference can help answer questions such as these:

- How does the choice of recommendation strategy affect user engagement, such as CTRs, purchase rates, and satisfaction ratings?
- What is the optimal recommendation strategy for a given user segment, item category, or context?

- What are the long-term effects of a recommendation strategy on user retention, loyalty, and lifetime value?

We'll round out this chapter by introducing one aspect of causal inference important to recommender systems, based on the concept of propensity score. We'll introduce propensity to quantify the adjusted likelihood of some items being shown to the user. We'll then see how this interacts with the famous Simpson's paradox.

Propensity

In many data science problems, we are forced to contend with confounders and, notably, the correlation between those confounders and a target outcome. Depending on the setting, the confounder may be of a variety of forms. Interestingly, in recommendation systems, that confounder can be the system itself! Offline evaluation of recommendation systems is subject to confounders derived from the item selection behavior of users and the deployed recommendation system.

If this issue seems a bit circular, it kind of is. This is sometimes called *closed-loop feedback*. One approach to mitigation is propensity weighting, which aims to address this problem by considering each feedback in the corresponding stratum based on the estimated propensities. You may recall that *propensity* refers to the likelihood of a user seeing an item; by inversely weighting by this, we can offset the selection bias. Compared to the standard offline holdout evaluation, this method attempts to represent the actual utility of the examined recommendation models.

One other approach to mitigating selection bias that we won't dive into is *counterfactual evaluation*, which estimates the actual utility of a recommendation model with propensity-weighting techniques more similar to off-policy evaluation approaches in reinforcement learning (RL). However, counterfactual evaluation often relies on accurate logging propensities in an open-loop setting where some random items are exposed to the user, which is not practical for most recommendation problems. If you have the option to include randomized recommendations to users for rating, this can help de-bias as well. One such setting where these methods may be combined is in RL-based recommenders that use explore-exploit methods like a multiarmed bandit or other structured randomization.

Inverse propensity scoring (IPS) is a propensity-based evaluation method that leverages importance sampling to account for the fact that the feedback collected from the deployed recommendation system is not uniformly random. The propensity score is a balancing factor that adjusts the observed feedback distribution conditioned on the propensity score. The IPS evaluation method is theoretically unbiased if open-loop feedback can be sampled from all possible items uniformly at random. In [Chapter 3](#), we discussed the Matthew effect, or “the rich get richer” for recommendation systems; IPS is one way to combat this effect. Note the relationship here between the two ideas of the Matthew effect and Simpson's paradox, when within different strata, selection effects create significant biasing.

Propensity weighting is based on the idea that the probability of an item being exposed to a user by the deployed recommendation system (the propensity score) affects the feedback that is collected from that user. By reweighting the feedback based on the propensity scores, we can adjust for the bias introduced by the deployed system and obtain a more accurate evaluation of the new recommendation model.

To apply IPS, we need to estimate the propensity scores for each item-user interaction in the collected feedback dataset. This can be done by modeling the probability that the deployed system would have exposed the item to the user at the time of the interaction. One simple approach is to use the popularity of the item as a proxy for its propensity score.

However, more sophisticated methods can be used to model the propen-

sity scores based on user and item features, as well as the context of the interaction.

Once the propensity scores are estimated, we can reweight the feedback by using importance sampling. Specifically, each feedback is weighted by the inverse of its propensity score so that items that are more likely to be exposed by the deployed system are downweighted, while items that are less likely to be exposed are upweighted. This reweighting process approximates a counterfactual distribution of feedback expected from surfacing recommendations from a uniform distribution of popularity.

Finally, we can use the reweighted feedback to evaluate the new recommendation model via standard metrics for evaluation, as we've seen in this chapter. The effectiveness of the new model is then compared to that of the deployed system by using the reweighted feedback, providing a fairer and more accurate evaluation of the new model's performance.

Simpson's and Mitigating Confounding

Simpson's paradox is predicated on the idea of a confounding variable that establishes strata within which we see (potentially misleading) co-variation. This paradox arises when the association between two variables is investigated but these variables are strongly influenced by a confounding variable.

In the case of recommendation systems, this confounding variable is the deployed model's characteristics and tendencies of selection. The propensity score is introduced as a measure of a system's deviation from an unbiased open-loop exposure scenario. This score allows for the design and analysis of offline evaluation of recommendation models based on the observed closed-loop feedback, mimicking some of the particular characteristics of the open-loop scenario.

Traditional descriptions of Simpson's paradox often suggest stratification, a well-known approach to identify and estimate causal effects by first identifying the underlying strata before investigating causal effects in each stratum. This approach enables the measurement of the potential outcome irrespective of the confounding variable. For recommendation

systems, this involves stratifying the observed outcome based on the possible values of the confounding variable, which is the deployed model’s characteristics.

The user-independent propensity score is estimated via a two-step generative process using the prior probability that an item is recommended by the deployed model and the conditional probability that the user interacts with the item, given that it is recommended. Based on a set of mild assumptions (but too mathematically technical to cover here), the user-independent propensity score can be estimated using maximum likelihood for each dataset.

We need to define the user-propensity score $p_{u,i}$, which indicates the tendency—or frequency—of the deployed model to expose item $i \in \mathcal{I}$ to user $u \in \mathcal{U}$. In practice, we marginalize over users to get the user-independent propensity score $p_{*,i}$. As described in [“Unbiased Offline Recommender Evaluation for Missing-Not-at-Random Implicit Feedback”](#) by Longqi Yang et al., the equation is as follows:

$$p_{*,i} \propto n_i^* \left(\frac{\gamma + 1}{2} \right)^{\frac{\gamma + 1}{2}}$$

Here, n_i^* is the total number of times item i interacted with, and γ is a parameter that affects the propensity distributions over items with different observed popularity. The power-law parameter γ affects the propensity distributions over items and depends on the examined dataset; we estimate the γ parameter by using maximum likelihood for each dataset.

With these estimates for propensity, we can then apply a simple inverse weighting $w_i = \frac{1}{p_i}$ when calculating the effect of feedback. Finally, we can combine these weightings with propensity matching, to generate counterfactual recommendations; by collecting approximately equal propensity items into strata, we can then use these strata as our confounding variable.

Doubly robust estimation (DRE) is a method that combines two models: one that models the probability of receiving the treatment (being recommended an item by the deployed model) and one that models the outcome of interest (the user's feedback on the item). The weights used in DRE depend on the predicted probabilities from both models. This method has the advantage that it can still provide unbiased estimates even if one of the models is misspecified.

The structural equations for a doubly robust estimator with propensity score weighting and outcome model is as follows:

$$\Theta = \frac{\sum w_i Y_i - f(X_i)}{\sum w_i T_i - p_i + \sum w_i p_i (1 - p_i)^2 f(X_i) - f^*(X_i))}$$

Here, Y_i is the outcome, X_i are covariates, T_i is the treatment, p_i is the propensity score, w_i is the weight, $f(X_i)$ is the outcome model, and $f^*(X_i)$ is the estimated outcome model.

For a great introduction to these considerations, check out [“Give Me a Robust Estimator—and Make It a Double!”](#).

Summary

What a whirlwind! Latent spaces are one of the *most* important aspects of recommendation systems. They are the representations that we use to encode our users and items. Ultimately, latent spaces are about more than dimension reduction; they are about understanding a geometry in which measures of distance encode the meaning relevant to your ML task.

The world of embeddings and encoders runs deep. We haven't had time to discuss CLIP embeddings (image + text) or the Poincaré disk (naturally hierarchical distance measures). We didn't dive deep into UMAP (a non-linear density-aware dimension-reduction technique) or HNSW (a method for retrieval in latent spaces that respects local geometry well). Instead, we point you to the (contemporaneously published) [article by Vicki Boykis](#) on embeddings, the [essay and guide to constructing embeddings](#) by Karel Minařík, or the [beautiful visual guide to text embeddings](#) by Meor Amer from Cohere.

We're now equipped with representations, but next we need to optimize. We're building *personalized* recommendation systems, so let's define the metrics that measure our performance on our task.